



THEEM COLLEGE OF ARTS, COMMERCE & SCIENCE

IKS PROJECT

On

Meru-Prastāra as Pascal Triangle and Dynamic Programming Model

Name : Roshan Mourya
Class : TYBSc (Computer Science)
Roll No. : 25
Subject : IKS
Academic Year : 2026 - 2027

Sr No	Section	Page No.
1	IKS Concept as CS Concept	3
2	Problem Statement	7
3	Conceptual Mapping Table	9
4	Algorithm Specification (Pseudocode)	11
5	Working Code	12
6	Minimum 10 Test Cases and Results	13
7	Complexity & Limitations	14
8	Conclusion	16

IKS Implementation Work

Topic

Meru-Prastāra as Pascal Triangle and Dynamic Programming Model

1. Title: IKS Concept as CS Concept

Meru-Prastāra as Pascal Triangle and Dynamic Programming Model

INTRODUCTION

Meru-Prastāra is an ancient Indian mathematical concept associated with the systematic study of combinations, patterns, and arrangements. The term *Meru-Prastāra* refers to a structured triangular arrangement and is traditionally connected with the study of poetic metres and combinatorial possibilities. Ancient Indian scholars developed methods for systematically representing different combinations of short and long syllables. This demonstrates that mathematical thinking was closely connected with other areas of knowledge such as language, poetry, music, and prosody.

The structure of Meru-Prastāra is particularly interesting from the perspective of modern mathematics and Computer Science because it has a strong structural connection with what is commonly known today as Pascal's Triangle. Pascal's Triangle is a triangular arrangement of numbers in which every row begins and ends with the number 1. The numbers in between are obtained by adding the two numbers directly above them.

For example, the first few rows can be represented as follows:

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
```

The construction of the triangle follows a simple but powerful mathematical rule. The first and last elements of every row are equal to 1. Every internal element is calculated by adding the two elements located directly above it. For example, in the row containing 1 3 3 1, the first internal value is obtained as $1 + 2 = 3$, while the second internal value is obtained as $2 + 1 = 3$. Therefore, the entire triangle can be constructed step by step from the previous row.

Mathematically, the elements of Pascal's Triangle represent binomial coefficients. The value at row n and position k can be represented as:

$$C(n,k) = n! / [k!(n-k)!]$$

For example:

$$C(4,2) = 4! / (2! \times 2!) = 6$$

Therefore, the row corresponding to $n = 4$ contains:

1 4 6 4 1

These coefficients have many applications in mathematics, probability, statistics, algebra, and Computer Science.

Meru-Prastāra and Combinatorics

One of the important aspects of Meru-Prastāra is its relationship with combinatorial arrangements. Combinatorics is the branch of mathematics concerned with counting and arranging objects according to specific rules. In the context of ancient Indian prosody, combinations of syllables could be studied systematically to determine different possible arrangements.

For example, if a poetic pattern consists of short and long syllables, different combinations can be generated. The number of possible arrangements increases as the number of positions increases. A triangular representation provides an organized way to understand these possibilities.

This idea is significant because modern Computer Science frequently deals with problems involving combinations and possible arrangements. Therefore, studying Meru-Prastāra provides an opportunity to understand how systematic mathematical reasoning can be connected to algorithmic problem solving.

Connection with Pascal's Triangle

The connection between Meru-Prastāra and Pascal's Triangle can be understood through their triangular structure and recurrence relationship. The important property is that an internal value is obtained from the sum of two values from the preceding row.

The recurrence relation can be written as:

$$P(n,k) = P(n-1,k-1) + P(n-1,k)$$

The boundary conditions are:

$$P(n,0) = 1$$

and

$$P(n,n) = 1$$

These simple rules are sufficient to generate the complete triangle.

For example:

```

      1
    1 1
  1 2 1
1 3 3 1
1 4 6 4 1

```

To calculate the value 6, we use the two values above it:

$$3 + 3 = 6$$

Similarly, the value 4 in the next row is calculated as:

$$1 + 3 = 4$$

and the other 4 is:

$$3 + 1 = 4$$

Thus, each new row depends on information that has already been calculated.

Connection with Dynamic Programming

This property creates a direct connection with Dynamic Programming, an important algorithmic technique in Computer Science.

Dynamic Programming is used when a problem can be divided into smaller subproblems and the solutions of those subproblems can be reused. Instead of solving the same subproblem multiple times, Dynamic Programming stores the result and uses it whenever required.

Pascal's Triangle provides a simple example of this principle. When calculating an element in a new row, the program uses values that were already calculated in the previous row.

For example:

Previous Row: 1 3 3 1

New Row: 1 4 6 4 1

The values are calculated as:

$$1 + 3 = 4$$

$$3 + 3 = 6$$

$$3 + 1 = 4$$

The program therefore does not need to calculate these values from the beginning every time. It can simply store the previous row and use it to generate the next row.

This approach is known as bottom-up Dynamic Programming, also called tabulation. The program starts with the smallest known values and progressively constructs larger solutions.

Importance in Computer Science

The implementation of Meru-Prastāra as Pascal's Triangle helps demonstrate several fundamental Computer Science concepts.

First, it demonstrates the use of arrays and lists for storing structured information. Each row can be represented as a list, while the complete triangle can be represented as a collection of lists.

Second, it demonstrates a recurrence relation. The value of the current element depends on two previously calculated elements.

Third, it demonstrates Dynamic Programming, because calculated values are stored and reused.

Fourth, it demonstrates algorithmic thinking. A mathematical rule is converted into a sequence of precise computational steps and then implemented using a programming language.

The overall process can therefore be represented as:

Ancient Indian Concept → Mathematical Pattern → Recurrence Relation → Algorithm →
Dynamic Programming → Computer Program

Purpose of This Project

The purpose of this project is not only to generate Pascal's Triangle using a computer program but also to demonstrate the relationship between an Indian mathematical tradition and a modern Computer Science concept.

Through this implementation, students can understand that mathematical ideas can be represented computationally. Meru-Prastāra provides an interesting example because its structured arrangement can be studied using modern concepts such as arrays, recurrence relations, iteration, recursion, and Dynamic Programming.

The project will therefore examine the concept of Meru-Prastāra, explain its relationship with Pascal's Triangle, develop an algorithm using Dynamic Programming, implement the algorithm in Python, test the program using multiple test cases, and analyze its computational complexity and limitations.

Overall, the study of Meru-Prastāra demonstrates how mathematical knowledge can provide a foundation for computational thinking. It also highlights the importance of understanding traditional Indian mathematical ideas and exploring their relevance to modern technology and Computer Science.

2. Problem Statement

2.1 Introduction

Meru-Prastara is a traditional Indian mathematical arrangement in which numbers are organized in the form of a triangle. The pattern is closely related to Pascal's Triangle and can be generated using a simple mathematical rule. This project aims to study Meru-Prastara from the perspective of Computer Science and understand how its method of construction relates to Dynamic Programming.

In Computer Science, many problems require repeated calculations. Performing the same calculation again and again can increase the execution time of a program. Dynamic Programming provides a solution to this problem by storing previously calculated results and reusing them when required.

Meru-Prastara provides a simple mathematical model to demonstrate this concept. Each inner element of a new row is obtained by adding two elements from the previous row. Therefore, the previously calculated values can be used to generate the next row.

2.2 Problem Definition

The main problem considered in this project is to generate Meru-Prastara for a given number of rows using an algorithm and implement the same concept using a programming language.

For example, if the user enters 5 rows, the expected output is:

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

The program must generate the triangle correctly by following the mathematical rule of Meru-Prastara.

2.3 Need for the Project

Generating a small triangle manually is easy, but calculating a large number of rows manually becomes time-consuming and can result in errors. A computer program can perform these calculations quickly and accurately.

- The project therefore focuses on developing a computational method that can:
- Accept the required number of rows from the user.
- Generate Meru-Prastara systematically.
- Use previously calculated values to generate new values.
- Display the complete triangular pattern.
- Demonstrate the relationship between Meru-Prastara and Pascal's Triangle.

- Explain how the concept can be related to Dynamic Programming.
- 2.4 Proposed Solution

The proposed solution is to develop an algorithm that generates each row using the values of the previous row.

- The basic approach is:
- Start with an empty triangle.
- Create the first row containing 1.
- For every new row, place 1 at the beginning and end.
- Calculate the middle elements by adding adjacent elements from the previous row.
- Store the newly generated row.
- Repeat the process until the required number of rows is generated.
- Display the complete Meru-Prastara.

This approach reduces unnecessary calculations because already generated values are used for creating the next row.

2.5 Objectives

- The main objectives of this project are:
- To understand the concept of Meru-Prastara.
- To study its relationship with Pascal's Triangle.
- To represent the mathematical concept as an algorithm.
- To implement Meru-Prastara using programming.
- To understand the role of previously calculated values.
- To demonstrate the basic concept of Dynamic Programming.
- To analyze the time and space requirements of the solution.
- To test the program using different input values.

2.6 Expected Outcome

The expected outcome of the project is a working program that correctly generates Meru-Prastara for the number of rows provided by the user.

The project will demonstrate that a traditional mathematical concept can be represented using modern computational techniques. It will also show how the reuse of previously calculated results can be used as a basic model for understanding Dynamic Programming.

Thus, the project establishes a practical connection between Indian Knowledge System, mathematics, algorithms, and Computer Science

3. Conceptual Mapping Table

This is an important section. It shows the connection between **IKS and Computer Science**.

Meru-Prastāra / IKS Concept	Computer Science Concept
Triangular arrangement	Data structure
First and last values are 1	Boundary condition
Middle values are obtained by addition	Recurrence relation
Previous row is used	Previous computed state
Reusing calculated values	Dynamic Programming
Number arrangement	Two-dimensional array
Generation of successive rows	Iterative algorithm
Combinatorial coefficients	Binomial coefficients
Meru-Prastāra	Pascal's Triangle
Mathematical pattern	Algorithmic pattern

Simple mapping:

Meru-Prastāra



Pascal's Triangle



Previous values are reused



Dynamic Programming



Computer Program



The architecture formalizes Pingala's ancient **Meru-Prastāra** as a canonical **Dynamic Programming (DP)** engine by transforming the geometric pyramid into a directed acyclic graph (DAG) evaluated via bottom-up tabulation. The boundary values of 1 on the outer flanks define the base cases ($T[n, 0] = T[n, n] = 1$), while the rule of summing adjacent cells from the preceding tier directly embodies the state transition recurrence ($T[n, k] = T[n-1, k-1] + T[n-1, k]$). By sweeping iteratively across levels, the system enforces a strict topological ordering that resolves overlapping subproblems exactly once, storing partial solutions in a 2D memory array (or a rolling 1D buffer) to collapse the computational complexity of binomial coefficient generation from an exponential $O(2^n)$ recursive tree down to a deterministic $O(n^2)$ polynomial runtime.

4. Algorithm Specification (Pseudocode)

Algorithm: Generate Meru-Prastāra using Dynamic Programming

START

1. Input the number of rows n .
2. Create an empty list called triangle.
3. For each row i from 0 to $n-1$:
 - a. Create a row containing $i+1$ elements.
 - b. Set the first element to 1.
 - c. Set the last element to 1.
 - d. For every middle element:
Calculate it using:
$$\text{current}[j] = \text{previous}[j-1] + \text{previous}[j]$$
 - e. Store the current row in triangle.
 - f. Display the current row.
4. Stop.

END

Example

For $n = 5$:

Row 0 $\rightarrow$ 1

Row 1 $\rightarrow$ 1 1

Row 2 $\rightarrow$ 1 2 1

Row 3 $\rightarrow$ 1 3 3 1

Row 4 $\rightarrow$ 1 4 6 4 1

5. Working Code

Here is a simple **Python** implementation

```
def generate_meru_prastara(n):  
    triangle = []  
    for i in range(n):  
        row = [1] * (i + 1)  
        for j in range(1, i):  
            row[j] = triangle[i - 1][j - 1] + triangle[i - 1][j]  
        triangle.append(row)  
    return triangle  
  
# Input  
n = int(input("Enter number of rows: "))  
  
# Generate Meru-Prastara  
triangle = generate_meru_prastara(n)  
  
# Display triangle  
print("\nMeru-Prastara (Pascal's Triangle):")
```

```
for row in triangle:
```

```
    print(row)
```

Sample Input

Enter number of rows: 5

Sample Output

Meru-Prastara (Pascal's Triangle):

[1]

[1, 1]

[1, 2, 1]

[1, 3, 3, 1]

[1, 4, 6, 4, 1]

6. Minimum 10 Test Cases

Test Case	Input (Rows)	Expected Output	Status
1	1	[1]	Pass
2	2	[1], [1,1]	Pass
3	3	[1], [1,1], [1,2,1]	Pass
4	4	Last row [1,3,3,1]	Pass
5	5	Last row [1,4,6,4,1]	Pass
6	6	Last row [1,5,10,10,5,1]	Pass
7	7	Last row [1,6,15,20,15,6,1]	Pass
8	8	Last row [1,7,21,35,35,21,7,1]	Pass
9	9	Last row [1,8,28,56,70,56,28,8,1]	Pass
10	10	Last row [1,9,36,84,126,126,84,36,9,1]	Pass

Special test cases

Input: 0

No rows to generate.

Input: 1

[1]

Input: 5

[1]

[1, 1]

[1, 2, 1]

[1, 3, 3, 1]

[1, 4, 6, 4, 1]

7. Complexity & Limitations

Time Complexity

Time complexity describes how the amount of computation increases when the input size increases. In this project, the input is the number of rows n that the program has to generate.

If the program generates n rows, the number of elements in each row is:

- Row 1 $\rightarrow$ 1 element
- Row 2 $\rightarrow$ 2 elements
- Row 3 $\rightarrow$ 3 elements
- ...
- Row $n \rightarrow n$ elements

Therefore, the total number of elements generated is:

$$1 + 2 + 3 + \dots + n$$

This can be represented using the formula:

$$n(n + 1) / 2$$

For example, if $n = 100$, the total number of elements generated is:

$$1 + 2 + 3 + \dots + 100 = 5050 \text{ elements}$$

The program therefore performs a number of operations proportional to the square of the number of rows. Hence, the Time Complexity is $O(n^2)$.

This means that when the number of rows increases, the amount of computation also increases. For a small number of rows, the program executes very quickly. However, for a very large number of rows, generating and processing all the elements requires more time and computational resources.

Space Complexity

Space complexity describes the amount of memory required by the program during execution.

In this implementation, the complete Meru-Prastāra/Pascal's Triangle is stored in the triangle list. If n rows are generated, the total number of stored elements is:

$$1 + 2 + 3 + \dots + n = n(n + 1) / 2$$

Therefore, the amount of memory required also increases approximately with the square of the number of rows.

Hence, the Space Complexity is $O(n^2)$.

For example, when $n = 100$, the program stores 5050 elements. As the value of n becomes larger, the memory required to store the complete triangle also increases.

The use of Dynamic Programming makes the program efficient in terms of avoiding repeated calculations because previously calculated values are reused. However, storing all the generated rows still requires $O(n^2)$ memory.

Limitations

Although the program successfully demonstrates the relationship between Meru-Prastāra, Pascal's Triangle, and Dynamic Programming, it has some limitations:

1. High Memory Requirement for Large Inputs

When a very large number of rows is generated, the program needs to store a large number of elements. Since the complete triangle is stored, the memory requirement increases as $O(n^2)$. Therefore, generating extremely large triangles may require significant memory.

2. Very Large Numerical Values

The numbers in Pascal's Triangle increase rapidly as the number of rows increases. For larger values of n , some elements can become extremely large. Processing and storing these large numbers may require additional computational resources.

3. Increased Computational Requirements

Although the algorithm reuses previously calculated values, generating a complete triangle still requires processing every element. As the number of rows increases, both the number of calculations and the amount of memory required increase.

4. Limited Historical Coverage

The main purpose of this project is to demonstrate the computational representation of Meru-Prastāra and its relationship with modern Computer Science. Therefore, the implementation does not cover every historical, cultural, or mathematical aspect of the original Indian mathematical tradition.

Overall Analysis

The project provides a simple and practical example of how an ancient mathematical pattern can be converted into a modern computational model. The $O(n^2)$ time complexity results from generating all elements of the triangle, while the $O(n^2)$ space complexity results from storing all rows.

Despite these limitations, the approach is suitable for demonstrating important Computer Science concepts such as arrays, recurrence relations, iteration, state reuse, and Dynamic Programming.

8. Conclusion

This project demonstrates the connection between the ancient Indian mathematical concept of Meru-Prastāra and the modern Computer Science concepts of Pascal's Triangle and Dynamic Programming. Meru-Prastāra follows a triangular pattern in which the boundary values are 1 and the inner values are obtained by adding values from the previous row.

The same mathematical structure can be efficiently implemented using Dynamic Programming because previously calculated values are stored and reused to calculate new values. The Python program successfully generates Meru-Prastāra for a given number of rows.

This implementation shows how an ancient Indian mathematical idea can be connected with modern algorithmic thinking. It also demonstrates the importance of mathematical patterns, recurrence relations, data structures, and Dynamic Programming in Computer Science.